MATERIAL SEMANA 04

O Poder do `useEffect`, Ciclo de Vida e Persistência no Navegador

Projeto Prático: Manutencao360 — Sistema de Gestão da Manutenção

Instituição: SENAI-SP - Serviço Nacional de Aprendizagem Industrial
Curso: Técnico em Desenvolvimento de Sistemas
Unidade Curricular: Desenvolvimento de Frameworks - React
Semana: 04
Projeto: Manutencao360 - Gestão de ativos, ordens de serviço e histórico de manutenção
Carga sugerida: 3h45min (5 aulas de 45 minutos)
Nível: Intermediário — Efeitos colaterais, temporizadores, ciclo de vida e persistência

---

Roteiro de estudos da semana

Este material foi planejado para guiar seu aprendizado autônomo, suas práticas de laboratório e a entrega do **Milestone 1 (Marco 1: Frontend Homologado)** no Google Classroom. Ele está estruturado em quatro etapas progressivas:

1. **Compreender:** descubra a fronteira entre renderização pura e efeitos colaterais (*side effects*), dominando a anatomia do hook `useEffect` e o papel crítico do array de dependências.
2. **Controlar e Limpar:** manipule o ciclo de vida do componente (montagem, atualização e desmontagem), orquestre temporizadores com `setInterval` e evite vazamentos de memória (*memory leaks*) com funções de limpeza (*cleanup*).
3. **Persistir com Segurança:** integre o `localStorage` do navegador com inicialização preguiçosa (*lazy state*) e tratamento defensivo de erros (`try...catch`), sincronizando o status de execução com a aba do navegador (`document.title`).

4. **Humanizar e Homologar:** estruture múltiplos estados visuais de interface (*loading*, *empty* e *error*), aplique requisitos práticos de acessibilidade (WCAG / a11y) com Lighthouse 90+ e gere sua tag de versão semântica no Git.

---

1. Como estudar este material

Para extrair o melhor rendimento do seu tempo de estudo:

1. **Digite o código no seu editor (VS Code):** evite copiar e colar. Digitar a sintaxe dos hooks e seus callbacks desenvolve raciocínio e memória motora.
2. **Inspecione o React DevTools e o Console (F12):** observe os componentes montando, os estados sendo atualizados e confira na aba *Application* > *Local Storage* as informações sendo gravadas em tempo real.
3. **Pratique Código Limpo (Clean Code):** mantenha funções com até 20 linhas e atribua responsabilidade única para cada função e componente (princípio SOLID).
4. **Comente sempre em português (pt-BR):** documente a intenção das regras de negócio e a justificativa dos efeitos colaterais.
5. **Realize testes manuais iterativos:** valide o temporizador antes de ligar o storage; teste a limpeza do timer antes de vincular atalhos de teclado.

---

2. Contextualização: Por que o Manutencao360?

Nas semanas anteriores, exploramos JSX, componentes, props e o hook `useState`. No entanto, aplicações reais de engenharia de software exigem desafios que vão além de adicionar e remover itens de uma lista em memória:

1. **Temporizadores de execução:** registrar com precisão o tempo gasto em cada serviço industrial sem que o relógio acelere loucamente ao mudar de aba.
2. **Sincronização com o navegador:** exibir o tempo decorrido e o equipamento diretamente no título da aba (`document.title`) para que o técnico possa consultar o status enquanto navega em outros manuais.
3. **Persistência automática:** garantir que ordens de serviço concluídas fiquem salvas no navegador mesmo após fechar a aba ou recarregar a página com `F5`.
4. **Resiliência e acessibilidade:** fornecer telas de carregamento (*loading*), mensagens orientadoras quando não houver registros (*empty state*) e acessibilidade por teclado para operadores industriais.

Assim nasce o **Manutencao360**:

None

```
+-----------------------------------------------------------------
---------+
|  (00:18:42) 🔧 Em execução | Bomba hidráulica EQ-014
[DevTools]|
+-----------------------------------------------------------------
---------+
|   MANUTENCAO360 — GESTÃO DA MANUTENÇÃO
|
|   "Acompanhe ativos, ordens de serviço e tempos de
intervenção."                  |
+-----------------------------------------------------------------
---------+   |
|      |  ORDEM EM EXECUÇÃO: [ Bomba hidráulica - EQ-014 ]
|
+-----------------------------------------------------------------
-----+      |
|      |                            00 : 18 : 42
|      |                Troca de vedação e inspeção preventiva
|      |
|
|[ ▶️ INICIAR (Espaço) ] [ ⏸️ PAUSAR ] [ ✓ CONCLUIR ORDEM ] [
ZERAR] |
+-----------------------------------------------------------------
-----+      +---------------------------------+  |
| | 📋 ORDENS CONCLUÍDAS (2)              |  | 🏭 DADOS DA ORDEM
| | • EQ-014 - Troca de vedação - 18 min  |  | Equipamento: Bomba
EQ-014       | | • EQ-008 - Inspeção elétrica - 32 min |  |
Responsável: Carlos Silva
| | Gravado no localStorage do navegador  |  | Atalhos: Espaço
(Play) / Esc
+-----------------------------------------------------------------
---------+
```

---

3. Objetivos de aprendizagem

Ao concluir os estudos deste guia, você será capaz de:

- diferenciar componentes puros de renderização de operações que provocam efeitos colaterais (*side effects*);
- dominar a sintaxe e o fluxo de execução do hook `useEffect`;
- controlar o momento exato de disparo dos efeitos através das 3 variações do array de dependências;
- diagnosticar, prevenir e neutralizar loops infinitos de renderização (*infinite render loops*);
- implementar a função de limpeza (*cleanup*) para descartar temporizadores (`clearInterval`) e listeners globais;
- sincronizar a aba do navegador (`document.title`) com variáveis de estado reativas;
- escutar eventos globais de teclado (`keydown`) com tratamento de cancelamento defensivo;
- serializar e desserializar coleções em formato JSON no `localStorage` do navegador;
- aplicar a técnica de inicialização preguiçosa (*lazy state initialization*) no `useState`;
- encapsular leituras e gravações em blocos defensivos `try...catch`;
- estruturar componentes dedicados a múltiplos estados visuais de tela (*loading*, *empty* e *error*);
- aplicar normas de acessibilidade digital (a11y): `<label htmlFor>`, `role="timer"`, `aria-label`, foco por teclado (`:focus-visible`) e auditoria Lighthouse 90+;
- homologar o **Milestone 1** no Git com commits semânticos e tag de versão (`v0.1.0-frontend`).

---

4. Conceitos Fundamentais

4.1 Função pura vs. Efeito colateral (*Side Effect*)

No modelo mental do React:

- O componente funcional deve ser uma **função pura**: dadas as mesmas `props` e o mesmo `state`, ele deve retornar sempre a mesma marcação JSX, sem alterar variáveis globais ou manipular elementos externos.

```
None
FLUXO PURO DO REACT:
(props, state) ⟶ [ Componente Funcional ] ⟶ JSX (Virtual DOM)
```

No entanto, sistemas de software precisam interagir com o mundo real:

- Disparar temporizadores (`setInterval`, `setTimeout`);
- Persistir dados no `localStorage`;
- Modificar o título da página (`document.title`);
- Escutar cliques e teclas na janela global (`window.addEventListener`);
- Efetuar requisições HTTP para servidores externos.

Todas essas operações externas são denominadas **Efeitos Colaterais (*Side Effects*)**.

Regra Essencial

Nunca execute efeitos colaterais soltos no corpo principal da função do componente! O corpo do componente é executado repetidamente a cada renderização. Executar temporizadores ou gravações soltas causará lentidão, multiplicações descontroladas de processos e congelamento da aplicação.

4.2 A anatomia do `useEffect`

O React fornece o hook `useEffect` como o local seguro e padronizado para executar efeitos colaterais após a renderização ter sido desenhada na tela:

```javascript
import { useEffect } from "react";

useEffect(() => {
  // 1. CÓDIGO DO EFEITO: roda após a renderização no navegador

  return () => {
    // 2. FUNÇÃO DE LIMPEZA (opcional): desfaz recursos antes do próximo ciclo
  };
}, [/* 3. ARRAY DE DEPENDÊNCIAS */]);
```

4.3 As 3 variações do Array de Dependências

O segundo parâmetro do `useEffect` determina com precisão **quando** a função do efeito deve ser executada:

Declaração	Quando executa?	Cenário típico de uso	Cuidado / Atenção
`useEffect(() => { ... })` *(Sem array)*	A **cada renderização** do componente.	Depuração detalhada de renderização (*logging*).	**Perigo de loop infinito** caso altere estado internamente!

`useEffect(() => { ... }, [])` *(Array vazio)*	**Uma única vez**, logo após a montagem do componente na tela.	Leitura inicial de dados, registro de atalhos de teclado globais.	Não utilize variáveis mutáveis sem declará-las, sob risco de leituras obsoletas (*stale closure*).
`useEffect(() => { ... }, [ativo, tempo])` *(Com dependências)*	Na montagem e **toda vez que qualquer dependência listada mudar**.	Atualização do `document.title`, gravação no `localStorage`.	Declare **todas** as variáveis de estado ou props referenciadas dentro do efeito.

4.4 O perigo do Loop Infinito e como evitá-lo

Veja o contraexemplo abaixo para compreender a mecânica de um laço infinito:

```javascript
// ERRO GRAVE: Este código causa travamento do navegador!
function MonitorComFalha() {
  const [contador, setContador] = useState(0);

  // Sem array de dependências e alterando o próprio estado dentro do efeito:
  useEffect(() => {
    setContador(contador + 1); // Altera estado -> Re-renderiza -> Roda efeito -> Altera estado...
  });

  return <div>Contador: {contador}</div>;
}
```

O React interrompe essa execução com a mensagem:
`Maximum update depth exceeded. This can happen when a component repeatedly calls setState inside useEffect...`

Regra prática: se o efeito altera um estado, esse mesmo estado **não** deve constar sem critério no array de dependências sem uma condição de parada clara.

4.5 Ciclo de vida e Função de Limpeza (*Cleanup*)

Em componentes funcionais, o ciclo de vida resume-se a três fases gerenciadas com elegância pelo `useEffect`:

```
None
CICLO DE VIDA DO COMPONENTE:

1. MONTAGEM (Mount):
   useEffect(() => { ... }, [])
   -> "O componente foi desenhado na tela pela primeira vez."

2. ATUALIZAÇÃO (Update):
   useEffect(() => { ... }, [isActive, elapsedSeconds])
   -> "Uma variável monitorada mudou. O efeito roda novamente."

3. DESMONTAGEM (Unmount) ou PRÉ-REEXECUÇÃO:
   return () => { clearInterval(id); }
   -> "O componente vai sumir da tela ou o efeito será
reiniciado. Limpe recursos!"
```

O que é um "Timer Fantasma" (Memory Leak)?

Se chamarmos `setInterval(...)` dentro de um componente sem cancelar a contagem anterior com `clearInterval(...)`, o temporizador continuará ativo em segundo plano na memória do navegador. A cada renderização, um novo temporizador concorrente será criado, fazendo com que o relógio passe a pular de 2 em 2 ou de 5 em 5 segundos!

```javascript
// Implementação segura com função de limpeza
useEffect(() => {
  if (!isActive) return undefined;

  // Cria o intervalo de 1 em 1 segundo
  const timerId = setInterval(() => {
    setElapsedSeconds((prev) => prev + 1);
  }, 1000);

  // Limpeza obrigatória: cancela o timer ao pausar ou desmontar
  return () => clearInterval(timerId);
}, [isActive, setElapsedSeconds]);
```

4.6 Sincronização com o Navegador (`document.title` e Atalhos de Teclado)

O `useEffect` é a ponte ideal para conectar sua aplicação com as APIs nativas do objeto `window` e `document`.

Sincronizando o título da aba:

```javascript
useEffect(() => {
  const status = isActive ? "Em andamento" : "Pausada";
  document.title = `${status} (${elapsedSeconds}s) | Manutencao360`;

  // Restaura o título padrão quando o componente desmontar
  return () => {
    document.title = "Manutencao360 - Gestão da Manutenção";
  };
}, [isActive, elapsedSeconds]);
```

Escutando atalhos globais de teclado:

```javascript
useEffect(() => {
  function handleKeyDown(event) {
    // Evita acionar atalhos enquanto o usuário digita em formulários
    if (["INPUT", "TEXTAREA", "SELECT"].includes(event.target.tagName)) return;

    if (event.code === "Space") {
      event.preventDefault(); // Impede a rolagem padrão da página
      setIsActive((prev) => !prev);
    }
  }

  window.addEventListener("keydown", handleKeyDown);
  return () => window.removeEventListener("keydown", handleKeyDown);
```

```
}, []);
```

4.7 Persistência com `localStorage` e Inicialização Preguiçosa (*Lazy State*)

O `localStorage` é uma área de armazenamento permanente por domínio oferecida pelo navegador (capacidade média de 5MB) que aceita exclusivamente cadeias de texto (*strings*).

```javascript
// Gravando: converte array/objeto para texto JSON
localStorage.setItem("minha_chave", JSON.stringify(dados));

// Lendo: recupera texto e converte de volta para array/objeto
const dados = JSON.parse(localStorage.getItem("minha_chave"));
```

O problema da leitura síncrona repetitiva:

Se passarmos `JSON.parse(localStorage.getItem(...))` diretamente no valor inicial do `useState`, a leitura no disco do navegador acontecerá a **cada segundo** de renderização do cronômetro, gerando gargalo desnecessário.

A Solução Profissional: Inicialização Preguiçosa (*Lazy State Initialization*)

Passar uma função de callback para o `useState` garante que a leitura e a conversão do JSON só ocorram **uma única vez**, no nascimento do componente:

```javascript
const [orders, setOrders] = useState(() => {
  try {
    const saved =
localStorage.getItem("manutencao360_ordens_v1");
    const parsed = saved ? JSON.parse(saved) : [];
    return Array.isArray(parsed) ? parsed : [];
  } catch (error) {
    console.error("Falha ao recuperar dados do storage:", error);
    return [];
  }
});
```

Por que usar `try...catch`?

Caso os dados do navegador sejam corrompidos por intervenção externa no DevTools ou a navegação anônima bloqueie o acesso, o bloco `try...catch` impede que a aplicação quebre com tela branca, iniciando graciosamente com um array vazio.

4.8 Arquitetura de Múltiplos Estados Visuais (Loading, Empty e Error)

Interfaces profissionais oferecem feedback imediato e transparente para qualquer condição operacional:

```
None
ESTADOS VISUAIS DO SISTEMA:
1. LOADING ──────► "Carregando histórico de ordens..." (Feedback de espera)
2. EMPTY ─────────► "Nenhuma ordem concluída ainda." (Orientação de primeiro uso)
3. ERROR ─────────► "Falha ao gravar no armazenamento local." (Recuperação e reset)
4. DATA (Normal) ► Listagem interativa das intervenções registradas
```

4.9 Acessibilidade na Prática (a11y / WCAG)

Garantir acessibilidade digital é dever técnico e ético do desenvolvedor de software:

1. **`role="timer"`:** informa às tecnologias assistivas e leitores de tela que aquele elemento exibe uma contagem de tempo contínua.
2. **`aria-label`:** descreve textualmente o objetivo de botões com ícones ou sem texto visível descritivo (ex.: `aria-label="Pausar ordem de serviço"`).
3. **`htmlFor`:** vincula semanticamente a tag `<label>` ao `<input id="...">`, permitindo que o leitor de tela identifique o propósito do campo e aumente a área clicável.
4. **Foco Visível (`:focus-visible`):** preserva uma borda contrastante e nítida quando a navegação ocorre via teclado (`Tab`), sem poluir a navegação por mouse.

Exercícios de Fixação Conceitual

Questão 1

Analise o trecho de código abaixo:

```javascript
useEffect(() => {
  console.log("Componente montado com sucesso.");
}, []);
```

Com base no segundo parâmetro fornecido, assinale a afirmação correta:

- (A) O efeito executará a cada clique em qualquer botão do componente.
- (B) O efeito executará uma única vez, logo após o componente ser montado na tela.
- (C) O efeito executará sempre que a variável `console` for alterada.
- (D) O efeito nunca será executado, pois o array de dependências está vazio.

Resposta comentada: **(B)**. Um array de dependências vazio `[]` indica ao React que o efeito não depende de nenhuma prop ou estado, disparando exclusivamente após a montagem inicial.

Questão 2

Por que devemos sempre retornar uma função de limpeza (*cleanup function*) ao iniciar um `setInterval` dentro do `useEffect`?

- (A) Porque sem a função de limpeza o React impede a compilação do código.
- (B) Para evitar que o temporizador continue executando em memória após o componente ser pausado ou removido, evitando vazamento de memória (*memory leak*).
- (C) Para salvar automaticamente o tempo decorrido no banco de dados.
- (D) Para forçar o navegador a reiniciar a aplicação em caso de erro.

Resposta comentada: **(B)**. Sem a limpeza via `clearInterval`, temporizadores antigos permanecem vivos na memória do navegador, acumulando instâncias paralelas concorrentes.

---

5. Passo a Passo Prático: Construindo o Manutencao360

Siga o roteiro técnico abaixo para implementar a aplicação completa na sua máquina de desenvolvimento.

5.1 Estrutura de Arquivos do Projeto

Certifique-se de que sua pasta de trabalho siga a organização modular recomendada:

```
None
manutencao360/
├── src/
|   ├── components/
|   |   ├── Header.jsx                # Cabeçalho e identificação
da aplicação
|   |   ├── MaintenanceTimer.jsx     # Display do cronômetro com
role="timer"
|   |   ├── MaintenanceControls.jsx  # Botões de ação (Iniciar,
Pausar, Concluir)
|   |   ├── MaintenanceHistory.jsx  # Listagem das ordens salvas
no storage
|   |   └── MaintenanceFeedback.jsx # Tratamento de Loading,
Empty e Error
|   ├── App.jsx                 # Orquestrador de estados e
efeitos
|   ├── App.css                 # Folha de estilos com
acessibilidade e foco
|   └── main.jsx                # Ponto de entrada do React
├── package.json
└── index.html
```

Passo 1: Criar o cabeçalho acessível (`src/components/Header.jsx`)

Crie o arquivo `src/components/Header.jsx`:

```javascript
// src/components/Header.jsx

// Cabecalho principal com identificacao semantica do sistema
function Header() {
  return (
    <header className="app-header">
      <div className="badge">Manutencao360 v1.0</div>
      <h1>Gestão de Ordens de Manutenção</h1>
```

```
      <p>Acompanhe equipamentos, intervenções e tempos de serviço
com precisão.</p>
    </header>
  );
}

export default Header;
```

Passo 2: Criar o display com timer e cleanup (`src/components/MaintenanceTimer.jsx`)

Crie o arquivo `src/components/MaintenanceTimer.jsx`:

JavaScript

```
// src/components/MaintenanceTimer.jsx
import { useEffect } from "react";

// Formata segundos inteiros para o formato hh:mm:ss
function formatTime(totalSeconds) {
  const hours = String(Math.floor(totalSeconds /
3600)).padStart(2, "0");
  const minutes = String(Math.floor((totalSeconds % 3600) /
60)).padStart(2, "0");
  const seconds = String(totalSeconds % 60).padStart(2, "0");
  return `${hours}:${minutes}:${seconds}`;
}

// Display do cronometro com limpeza automatica de intervalo
function MaintenanceTimer({ elapsedSeconds, setElapsedSeconds,
isActive }) {
  useEffect(() => {
    if (!isActive) return undefined;

    const timerId = setInterval(() => {
      setElapsedSeconds((prev) => prev + 1);
    }, 1000);
```

```
    // Funcao de limpeza para descartar o timer ao pausar ou
desmontar
    return () => clearInterval(timerId);
  }, [isActive, setElapsedSeconds]);

  return (
    <div className="timer-display" role="timer" aria-label="Tempo
de execução da ordem">
      <span
className="digits">{formatTime(elapsedSeconds)}</span>
    </div>
  );
}

export default MaintenanceTimer;
```

Passo 3: Criar os botões de controle (`src/components/MaintenanceControls.jsx`)

Crie o arquivo `src/components/MaintenanceControls.jsx`:

JavaScript

```
// src/components/MaintenanceControls.jsx

// Botoes de acao da ordem com rotulos e descricoes de
acessibilidade
function MaintenanceControls({
  isActive,
  onToggleActive,
  onReset,
  onComplete,
  canStart,
  canComplete
}) {
  return (
    <div className="controls-group">
      <button
```

```
        type="button"
        className={`btn-primary ${isActive ? "btn-pause" :
"btn-start"}`}
        onClick={onToggleActive}
        disabled={!isActive && !canStart}
        aria-label={isActive ? "Pausar ordem de manutenção" :
"Iniciar ordem de manutenção"}
      >
        {isActive ? "Pausar" : "Iniciar"}
      </button>

      <button
        type="button"
        className="btn-secondary"
        onClick={onReset}
        aria-label="Zerar o tempo registrado nesta ordem"
      >
        Zerar
      </button>

      <button
        type="button"
        className="btn-success"
        onClick={onComplete}
        disabled={!canComplete}
        aria-label="Concluir ordem e salvar no histórico"
      >
        Concluir ordem
      </button>
    </div>
  );
}

export default MaintenanceControls;
```

Passo 4: Criar o histórico persistido (`src/components/MaintenanceHistory.jsx`)

Crie o arquivo `src/components/MaintenanceHistory.jsx`:

JavaScript

```
// src/components/MaintenanceHistory.jsx

// Lista de ordens concluidas recuperadas do localStorage
function MaintenanceHistory({ orders = [], onClearHistory }) {
  if (orders.length === 0) return null;

  return (
    <section className="history-section"
aria-labelledby="history-title">
      <div className="history-header">
        <h2 id="history-title">Ordens de manutenção concluídas
({orders.length})</h2>
        <button
          type="button"
          className="btn-clear"
          onClick={onClearHistory}
          aria-label="Limpar todo o histórico de ordens
concluídas"
        >
          Limpar histórico
        </button>
      </div>

      <ul className="history-list">
        {orders.map((item) => (
          <li key={item.id} className="history-card">
            <div className="card-info">
              <strong>{item.equipment}</strong>
              <span className="badge-mode">{item.service}</span>
              <small className="card-tech">Técnico:
{item.technician}</small>
            </div>
            <div className="card-meta">
              <span className="card-duration">
                {Math.floor(item.durationSeconds / 60)} min
({item.durationSeconds}s)
              </span>
```

```
              <time
className="card-time">{item.completedAt}</time>
            </div>
          </li>
        ))}
      </ul>
    </section>
  );
}

export default MaintenanceHistory;
```

Passo 5: Criar os estados visuais (`src/components/MaintenanceFeedback.jsx`)

Crie o arquivo `src/components/MaintenanceFeedback.jsx`:

JavaScript

```
// src/components/MaintenanceFeedback.jsx

// Componente responsavel por exibir os estados Loading, Empty e
Error
function MaintenanceFeedback({ isLoading, error, orderCount,
onResetStorage }) {
  if (isLoading) {
    return (
      <div className="feedback-container loading" role="status">
        <div className="spinner" aria-hidden="true"></div>
        <p>Carregando ordens de manutenção...</p>
      </div>
    );
  }

  if (error) {
    return (
      <div className="feedback-container error" role="alert">
        <p>⚠️ Ocorreu uma falha no armazenamento: {error}</p>
```

```
        <button type="button" onClick={onResetStorage}
className="btn-repair">
          Restaurar Dados e Limpar Armazenamento
        </button>
      </div>
    );
  }

  if (orderCount === 0) {
    return (
      <div className="feedback-container empty">
        <p className="empty-icon" aria-hidden="true">🔧</p>
        <h3>Nenhuma ordem concluída</h3>
        <p>Informe o equipamento e o serviço acima para iniciar o
primeiro registro.</p>
      </div>
    );
  }

  return null;
}

export default MaintenanceFeedback;
```

Passo 6: Orquestrar a aplicação no `src/App.jsx`

Substitua o conteúdo de `src/App.jsx`. Cada função atende aos padrões de Clean Code (máximo 20 linhas) e responsabilidade única:

JavaScript

```
// src/App.jsx
import { useState, useEffect } from "react";
import Header from "./components/Header";
import MaintenanceTimer from "./components/MaintenanceTimer";
import MaintenanceControls from
"./components/MaintenanceControls";
import MaintenanceHistory from "./components/MaintenanceHistory";
```

```
import MaintenanceFeedback from
"./components/MaintenanceFeedback";
import "./App.css";

const STORAGE_KEY = "manutencao360_ordens_v1";

// Cria o registro formatado da ordem concluida
function buildOrderRecord(equipment, service, technician,
duration) {
  return {
    id: Date.now().toString(),
    equipment: equipment.trim(),
    service: service.trim(),
    technician: technician.trim() || "Não informado",
    durationSeconds: duration,
    completedAt: new Date().toLocaleString([], {
      dateStyle: "short",
      timeStyle: "short"
    })
  };
}

// Recupera dados do localStorage com seguranca defensiva
function loadInitialOrders() {
  try {
    const saved = localStorage.getItem(STORAGE_KEY);
    const parsed = saved ? JSON.parse(saved) : [];
    return Array.isArray(parsed) ? parsed : [];
  } catch (readError) {
    console.error("Falha ao ler ordens do storage:", readError);
    return [];
  }
}

function App() {
  const [elapsedSeconds, setElapsedSeconds] = useState(0);
  const [isActive, setIsActive] = useState(false);
  const [equipmentName, setEquipmentName] = useState("");
```

```
  const [serviceDescription, setServiceDescription] =
useState("");
  const [technicianName, setTechnicianName] = useState("");
  const [isLoading, setIsLoading] = useState(true);
  const [error, setError] = useState(null);
  const [orders, setOrders] = useState(loadInitialOrders);

  // EFEITO 1: Simula inicializacao assincrona dos dados
  useEffect(() => {
    const timer = setTimeout(() => setIsLoading(false), 500);
    return () => clearTimeout(timer);
  }, []);

  // EFEITO 2: Sincroniza estado de orders com o localStorage
  useEffect(() => {
    try {
      localStorage.setItem(STORAGE_KEY, JSON.stringify(orders));
      setError(null);
    } catch (saveError) {
      setError("Não foi possível salvar os registros no
navegador.");
    }
  }, [orders]);

  // EFEITO 3: Atualiza o titulo da aba do navegador
  useEffect(() => {
    const status = isActive ? "Em execução" : "Pausada";
    const hours = String(Math.floor(elapsedSeconds /
3600)).padStart(2, "0");
    const minutes = String(Math.floor((elapsedSeconds % 3600) /
60)).padStart(2, "0");
    const seconds = String(elapsedSeconds % 60).padStart(2, "0");
    const item = equipmentName.trim() || "Nova ordem";

    document.title = `${status} ${hours}:${minutes}:${seconds} |
${item}`;
    return () => {
      document.title = "Manutencao360 - Gestão da Manutenção";
    };
```

```
  }, [elapsedSeconds, isActive, equipmentName]);

  // EFEITO 4: Atalhos de teclado (Espaco para alternar, Esc para
resetar)
  useEffect(() => {
    function handleKeyDown(event) {
      if (["INPUT", "TEXTAREA",
"SELECT"].includes(event.target.tagName)) return;
      if (event.code === "Space") {
        event.preventDefault();
        setIsActive((prev) => (prev ? false :
Boolean(equipmentName.trim() && serviceDescription.trim())));
      } else if (event.code === "Escape") {
        setIsActive(false);
        setElapsedSeconds(0);
      }
    }

    window.addEventListener("keydown", handleKeyDown);
    return () => window.removeEventListener("keydown",
handleKeyDown);
  }, [equipmentName, serviceDescription]);

  function handleToggleActive() {
    if (!equipmentName.trim() || !serviceDescription.trim())
return;
    setIsActive((prev) => !prev);
  }

  function handleReset() {
    setIsActive(false);
    setElapsedSeconds(0);
  }

  function handleCompleteOrder() {
    if (!equipmentName.trim() || !serviceDescription.trim() ||
elapsedSeconds === 0) return;
    const newRecord = buildOrderRecord(equipmentName,
serviceDescription, technicianName, elapsedSeconds);
```

```
    setIsActive(false);
    setOrders((prev) => [newRecord, ...prev]);
    setElapsedSeconds(0);
    setEquipmentName("");
    setServiceDescription("");
    setTechnicianName("");
  }

  function handleClearHistory() {
    setOrders([]);
    try {
      localStorage.removeItem(STORAGE_KEY);
      setError(null);
    } catch (removeError) {
      setError("Falha ao limpar o histórico local.");
    }
  }

  const canStart = Boolean(equipmentName.trim() &&
serviceDescription.trim());
  const canComplete = Boolean(canStart && elapsedSeconds > 0);

  return (
    <main className="app-container">
      <Header />

      <section className="timer-panel" aria-label="Ordem de
manutenção em execução">
        <div className="maintenance-form">
          <div className="field-group">
            <label htmlFor="equipment-input">Equipamento ou
código do ativo</label>
            <input
              id="equipment-input"
              type="text"
              placeholder="Ex: Bomba hidráulica EQ-014"
              value={equipmentName}
              onChange={(e) => setEquipmentName(e.target.value)}
              disabled={isActive}
```

```
            />
          </div>

          <div className="field-group">
            <label htmlFor="service-input">Serviço a
executar</label>
            <input
              id="service-input"
              type="text"
              placeholder="Ex: Troca de vedação e inspeção"
              value={serviceDescription}
              onChange={(e) =>
setServiceDescription(e.target.value)}
              disabled={isActive}
            />
          </div>

          <div className="field-group">
            <label htmlFor="technician-input">Técnico responsável
(opcional)</label>
            <input
              id="technician-input"
              type="text"
              placeholder="Ex: Carlos Silva"
              value={technicianName}
              onChange={(e) => setTechnicianName(e.target.value)}
              disabled={isActive}
            />
          </div>
        </div>

        <MaintenanceTimer
          elapsedSeconds={elapsedSeconds}
          setElapsedSeconds={setElapsedSeconds}
          isActive={isActive}
        />

        <MaintenanceControls
          isActive={isActive}
```

```
          onToggleActive={handleToggleActive}
          onReset={handleReset}
          onComplete={handleCompleteOrder}
          canStart={canStart}
          canComplete={canComplete}
        />
      </section>

      <MaintenanceFeedback
        isLoading={isLoading}
        error={error}
        orderCount={orders.length}
        onResetStorage={handleClearHistory}
      />

      {!isLoading && !error && (
        <MaintenanceHistory orders={orders}
onClearHistory={handleClearHistory} />
      )}
    </main>
  );
}

export default App;
```

Passo 7: Aplicar a folha de estilos acessível (`src/App.css`)

Substitua o conteúdo de `src/App.css` para aplicar layout profissional, responsividade e foco visível acessível:

CSS

```
/* src/App.css */
:root {
  --primary: #2563eb;
  --primary-hover: #1d4ed8;
  --secondary: #64748b;
  --success: #16a34a;
  --danger: #dc2626;
```

```
  --bg-main: #f8fafc;
  --bg-card: #ffffff;
  --text-main: #0f172a;
  --text-muted: #64748b;
  --border: #e2e8f0;
  --focus-ring: #2563eb;
}

body {
  margin: 0;
  font-family: system-ui, -apple-system, BlinkMacSystemFont,
"Segoe UI", Roboto, sans-serif;
  background-color: var(--bg-main);
  color: var(--text-main);
  line-height: 1.5;
}

.app-container {
  max-width: 760px;
  margin: 2rem auto;
  padding: 0 1rem;
}

.app-header {
  text-align: center;
  margin-bottom: 2rem;
}

.app-header h1 {
  font-size: 1.85rem;
  margin: 0.5rem 0 0.25rem;
}

.app-header p {
  color: var(--text-muted);
  margin: 0;
}

.badge {
```

```
  display: inline-block;
  padding: 0.25rem 0.65rem;
  font-size: 0.8rem;
  font-weight: 600;
  background-color: #dbeafe;
  color: var(--primary);
  border-radius: 9999px;
}

.timer-panel {
  background-color: var(--bg-card);
  border: 1px solid var(--border);
  border-radius: 12px;
  padding: 1.5rem;
  box-shadow: 0 4px 6px -1px rgba(0, 0, 0, 0.05);
}

.maintenance-form {
  display: grid;
  grid-template-columns: 1fr;
  gap: 1rem;
  margin-bottom: 1.5rem;
}

@media (min-width: 600px) {
  .maintenance-form {
    grid-template-columns: 1fr 1fr;
  }
  .maintenance-form .field-group:last-child {
    grid-column: span 2;
  }
}

.field-group {
  display: flex;
  flex-direction: column;
  gap: 0.35rem;
}
```

```css
.field-group label {
  font-size: 0.875rem;
  font-weight: 600;
  color: var(--text-main);
}

.field-group input {
  padding: 0.6rem 0.75rem;
  border: 1px solid var(--border);
  border-radius: 8px;
  font-size: 0.95rem;
}

.field-group input:disabled {
  background-color: #f1f5f9;
  cursor: not-allowed;
}

.timer-display {
  text-align: center;
  padding: 1.5rem 0;
  background-color: #0f172a;
  border-radius: 10px;
  margin-bottom: 1.5rem;
}

.timer-display .digits {
  font-family: ui-monospace, SFMono-Regular, Menlo, Monaco,
Consolas, monospace;
  font-size: 3rem;
  font-weight: 700;
  color: #38bdf8;
  letter-spacing: 2px;
}

.controls-group {
  display: flex;
  gap: 0.75rem;
  flex-wrap: wrap;
```

```
}

button {
  flex: 1;
  padding: 0.75rem 1rem;
  font-size: 0.95rem;
  font-weight: 600;
  border-radius: 8px;
  border: none;
  cursor: pointer;
  transition: background-color 0.15s ease, opacity 0.15s ease;
}

button:disabled {
  opacity: 0.5;
  cursor: not-allowed;
}

/* Acessibilidade WCAG: Foco visivel destacado por teclado */
button:focus-visible,
input:focus-visible {
  outline: 3px solid var(--focus-ring);
  outline-offset: 2px;
}

.btn-start {
  background-color: var(--primary);
  color: #ffffff;
}

.btn-start:hover:not(:disabled) {
  background-color: var(--primary-hover);
}

.btn-pause {
  background-color: #d97706;
  color: #ffffff;
}
```

```css
.btn-secondary {
  background-color: #e2e8f0;
  color: #334155;
}

.btn-success {
  background-color: var(--success);
  color: #ffffff;
}

.feedback-container {
  margin-top: 1.5rem;
  padding: 1.5rem;
  border-radius: 10px;
  text-align: center;
}

.feedback-container.loading {
  background-color: #eff6ff;
  color: var(--primary);
}

.spinner {
  width: 28px;
  height: 28px;
  border: 3px solid #bfdbfe;
  border-top-color: var(--primary);
  border-radius: 50%;
  animation: spin 0.8s linear infinite;
  margin: 0 auto 0.75rem;
}

@keyframes spin {
  to { transform: rotate(360deg); }
}

.feedback-container.empty {
  background-color: var(--bg-card);
  border: 1px dashed #cbd5e1;
```

```
  color: var(--text-muted);
}

.empty-icon {
  font-size: 2.2rem;
  margin: 0 0 0.5rem;
}

.feedback-container.error {
  background-color: #fef2f2;
  border: 1px solid #fecaca;
  color: var(--danger);
}

.btn-repair {
  margin-top: 0.75rem;
  background-color: var(--danger);
  color: #ffffff;
  padding: 0.5rem 1rem;
}

.history-section {
  margin-top: 2rem;
  background-color: var(--bg-card);
  border: 1px solid var(--border);
  border-radius: 12px;
  padding: 1.5rem;
}

.history-header {
  display: flex;
  justify-content: space-between;
  align-items: center;
  margin-bottom: 1rem;
}

.history-header h2 {
  font-size: 1.2rem;
  margin: 0;
```

```
}

.btn-clear {
  flex: none;
  background: none;
  color: var(--danger);
  padding: 0.35rem 0.65rem;
  font-size: 0.85rem;
}

.history-list {
  list-style: none;
  padding: 0;
  margin: 0;
  display: flex;
  flex-direction: column;
  gap: 0.75rem;
}

.history-card {
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 0.9rem 1rem;
  background-color: #f8fafc;
  border: 1px solid var(--border);
  border-radius: 8px;
}

.card-info {
  display: flex;
  flex-direction: column;
  gap: 0.2rem;
}

.badge-mode {
  display: inline-block;
  font-size: 0.75rem;
  color: var(--text-muted);
```

```
}

.card-tech {
  font-size: 0.8rem;
  color: #475569;
}

.card-meta {
  display: flex;
  flex-direction: column;
  align-items: flex-end;
  gap: 0.2rem;
}

.card-duration {
  font-weight: 600;
  color: var(--primary);
}

.card-time {
  font-size: 0.8rem;
  color: var(--text-muted);
}
```

---

6. Guia de Resolução de Problemas (Troubleshooting)

1. O cronômetro passa a contar de 2 em 2 ou de 3 em 3 segundos

- **Diagnóstico:** Falta da função de limpeza (*cleanup*). Cada vez que o componente re-renderizou, um novo `setInterval` foi instanciado sem cancelar o anterior.
- **Solução:** Certifique-se de que o `useEffect` do timer retorna obrigatoriamente:

```JavaScript
return () => clearInterval(timerId);
```

---

2. Ao recarregar a tela (`F5`), a aplicação quebra em branco com erro de JSON

- **Diagnóstico:** O valor armazenado na chave do `localStorage` foi editado com erro de sintaxe ou armazenou `"undefined"`.
- **Solução:** Sempre envolva a leitura com `try...catch` e use a inicialização preguiçosa:

```JavaScript
const [orders, setOrders] = useState(() => {
  try {
    const saved = localStorage.getItem(STORAGE_KEY);
    return saved ? JSON.parse(saved) : [];
  } catch {
    return [];
  }
});
```

---

3. O título da aba não atualiza ao iniciar ou pausar

- **Diagnóstico:** O array de dependências do efeito que atualiza o `document.title` não listou `isActive` ou `elapsedSeconds`.
- **Solução:** Revise o array de dependências e inclua todas as variáveis reativas utilizadas:

```JavaScript
useEffect(() => {
  // ...
}, [elapsedSeconds, isActive, equipmentName]);
```

---

7. Desafio de Entrega e Roteiro de Testes Manuais

Execute o roteiro completo de testes manuais na sua estação antes de registrar a entrega no Google Classroom:

Caso	Ação Realizada	Comportamento Esperado	Status
01	Tentar clicar em "Iniciar" com campos vazios	O botão permanece desabilitado; a ordem não inicia.	[ ]
02	Informar equipamento e serviço e clicar em "Iniciar"	O cronômetro começa a contar; o botão muda para "Pausar"; os campos são bloqueados.	[ ]
03	Alternar para outra aba do navegador	O título da aba exibe o tempo contando e o nome do equipamento.	[ ]
04	Pressionar a tecla `Espaço` fora de campos de texto	A contagem alterna instantaneamente entre em execução e pausada.	[ ]
05	Pressionar a tecla `Esc`	A contagem para e o tempo é zerado para 00:00:00.	[ ]
06	Contar alguns segundos e clicar em "Concluir ordem"	A ordem aparece no topo da lista com técnico, duração em minutos e data/hora.	[ ]
07	Pressionar `F5` (recarregar página inteira)	O histórico permanece preservado na tela (lido do `localStorage`).	[ ]
08	Navegar usando apenas a tecla `Tab`	O anel de foco visível em azul contorna todos os campos e botões ativos.	[ ]
09	Executar auditoria do Lighthouse (Aba Acessibilidade)	Pontuação de Acessibilidade atinge entre **95 e 100 pontos**.	[ ]

---

8. Versionamento Semântico e Homologação do Marco 1

Para homologar o **Milestone 1 (Marco 1: Frontend Concluído)**, execute os seguintes comandos no terminal do Git:

Shell

```
# 1. Verifique o status das modificações
git status

# 2. Adicione todos os arquivos modificados
git add .

# 3. Registre o commit semântico de homologação
git commit -m "feat: conclui milestone 1 com useEffect,
localStorage e acessibilidade wcag"

# 4. Crie a tag semântica do marco de frontend
git tag -a v0.1.0-frontend -m "Homologacao do Milestone 1:
Frontend finalizado"

# 5. Envie os commits e as tags para o GitHub
git push origin main
git push origin --tags
```

---

9. Tabela de Autoavaliação

Avalie seu nível de domínio prático sobre as competências desenvolvidas nesta semana:

Competência Desenvolvida na Semana 04	**Preciso praticar mais**	**Consigo com apoio**	**Domino com segurança**
Sei diferenciar funções puras de efeitos colaterais (*side effects*).			
Domino as 3 variações do array de dependências do `useEffect`.			
Compreendo e sei corrigir laços infinitos de renderização.			
Sei implementar a função de limpeza (*cleanup*) com `clearInterval`.			

Sei sincronizar a aba do navegador (`document.title`) com estados reativos.
Sei escutar atalhos de teclado no objeto global `window` com limpeza.
Domino a persistência e recuperação de JSON no `localStorage`.
Sei aplicar inicialização preguiçosa com `useState(() => ...)` e `try/catch`.
Sei projetar interfaces com múltiplos estados (*loading*, *empty* e *error*).
Sei aplicar acessibilidade prática: `role="timer"`, `<label htmlFor>` e `:focus-visible`.
Sei auditar a aplicação no Google Lighthouse e atingir nota 90+.

---

10. Glossário Técnico do Estudante

- **Cleanup Function (Função de Limpeza):** Função de retorno do `useEffect` executada antes de reiniciar o efeito ou na desmontagem do componente, liberando recursos ativos da memória.
- **Dependencies Array (Array de Dependências):** Lista de variáveis reativas observadas pelo React para decidir o momento de reexecutar um efeito.
- **Efeito Colateral (Side Effect):** Qualquer operação que modifique estados ou acesse sistemas externos à renderização visual do componente.
- **Lazy State Initialization (Inicialização Preguiçosa):** Passagem de uma função para o `useState` garantindo que cálculos ou acessos ao storage só rodem na primeira montagem.
- **Memory Leak (Vazamento de Memória):** Retenção desnecessária de processos ou objetos na memória que degrada o desempenho do navegador.
- **Role Timer:** Propriedade ARIA que comunica às tecnologias assistivas que determinado elemento visual representa um mostrador de tempo.
- **Stale Closure:** Efeito no JavaScript onde uma função captura e memoriza o valor antigo de uma variável, deixando de enxergar seu estado atual.
- **WCAG (Web Content Accessibility Guidelines):** Conjunto normativo internacional que orienta a construção de sites acessíveis para todas as pessoas.